

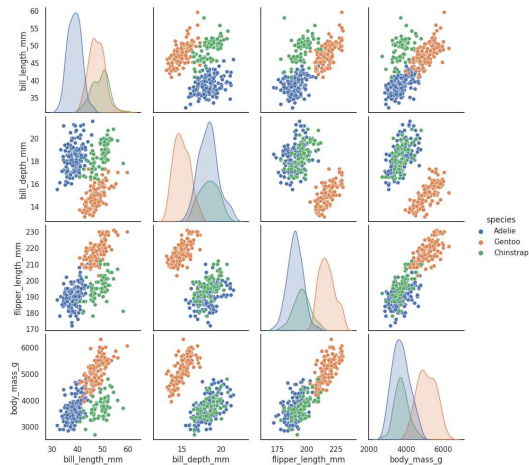
Projections and clusterings

AI for ecologists

Paul Tresson

Introduction

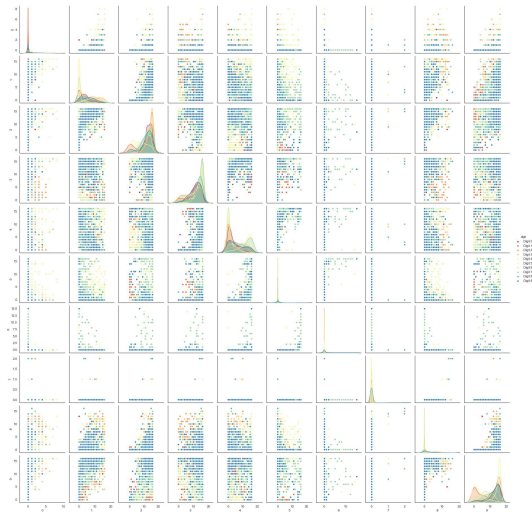
Visualize what is happening in higher dimensions



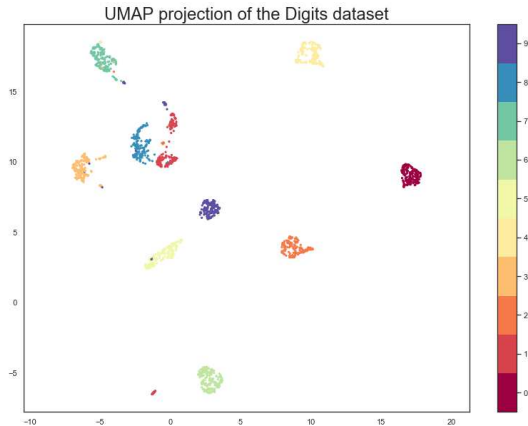
Visualize what is happening in higher dimensions

0 0 8 3 1 4 5 0 0 2 1 9 7 4 4 7 4 2 6 4
1 1 4 7 7 0 4 3 2 5 0 4 2 5 5 3 6 1 7 9
2 2 1 3 6 5 8 2 5 7 0 7 8 6 5 6 4 6 0
3 3 7 3 3 3 8 5 8 9 6 2 7 7 1 6 4 8
4 4 7 4 2 6 4 4 6 1 5 8 2 8 0 4 6 7 9
5 5 3 6 1 3 9 5 0 4 3 5 9 9 0 7 3 5 3
6 6 5 6 7 6 0 6 0 1 8 3 5 0 2 1 4 0
7 7 1 6 4 1 7 7 9 2 1 4 4 2 5 3 4 1
8 8 0 4 6 7 9 8 6 0 0 4 8 2 5 7 0 2
9 9 0 9 3 5 8 9 3 3 0 8 8 3 5 8 9 2 3
0 0 2 1 1 4 4 0 8 3 1 7 7 4 6 2 5 7 4
1 1 5 7 0 3 7 2 1 4 7 6 6 0 6 4 8 2 6
2 2 8 9 1 2 3 7 3 3 3 8 7 9 2 2 4 5 7
3 3 8 9 1 2 3 7 3 3 3 8 7 9 2 2 4 5 7
4 4 2 5 7 0 3 7 2 1 4 7 6 6 0 6 4 8 2 6
5 5 0 2 6 2 5 5 3 6 4 4 8 9 9 3 0 1 9
6 6 1 8 8 2 6 6 5 3 6 6 7 0 0 8 3 4 5 0
7 7 2 2 4 5 7 7 1 6 3 5 1 4 2 3 7 0 4 1
8 8 6 0 3 7 8 8 0 4 1 4 2 3 1 3 6 5 8 2
9 9 3 0 4 9 9 3 9 9 3 9 3 3 3 3 3 8 3

Visualize what is happening in higher dimensions



Visualize what is happening in higher dimensions

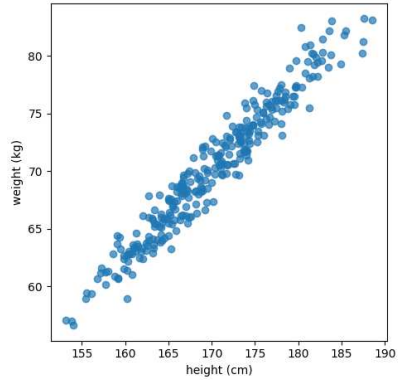


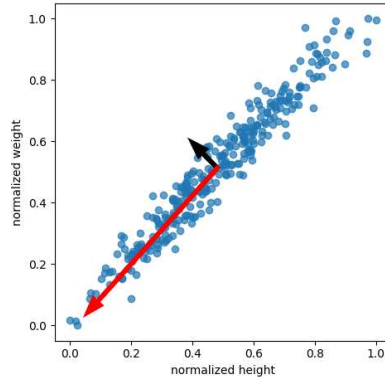
Projections

Projections

Principa Component Analysis

PCA





PCA calculation

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ x_3 & y_3 \\ \dots & \dots \end{bmatrix}$$

PCA calculation

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ x_3 & y_3 \\ \dots & \dots \end{bmatrix}$$

$$C = \frac{1}{n-1} X^T X$$

$$C = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix}$$

PCA calculation

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ x_3 & y_3 \\ \dots & \dots \end{bmatrix}$$

$$C = \frac{1}{n-1} X^T X$$

$$C = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix}$$

$$Cv = \lambda v$$

$$\begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \lambda \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

PCA calculation

$$X = \begin{bmatrix} x_1 & y_1 \\ x_2 & y_2 \\ x_3 & y_3 \\ \dots & \dots \end{bmatrix}$$

$$C = \frac{1}{n-1} X^T X$$

$$C = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix}$$

$$Cv = \lambda v$$

$$\begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{yx} & \sigma_y^2 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \lambda \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

$$P = \begin{bmatrix} v_x & v_y \end{bmatrix}$$

PCA calculation

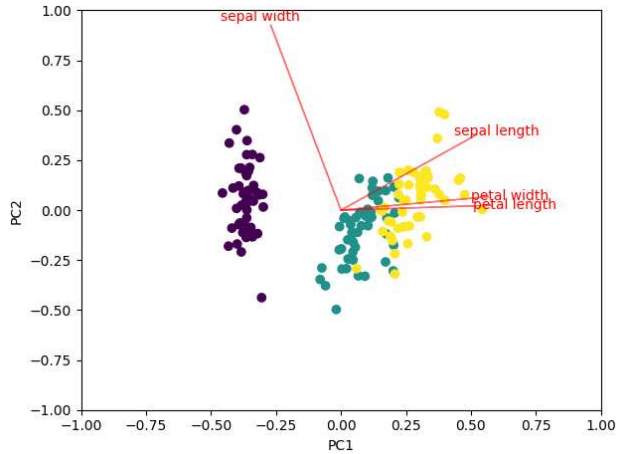
```
# normalize
normalized_data = (points - np.min(points, axis=0)) / (
    np.max(points, axis=0) - np.min(points, axis=0)
)

# get covariance
cov = np.cov(normalized_data, rowvar=False)

# calculate eigenvalues and eigenvectors of the covariance matrix
eigvals, eigvecs = np.linalg.eig(cov)

# scale eigenvectors
scaled_eigvecs = eigvecs * np.sqrt(eigvals)
```

PCA interpretability



PCA advantages and drawbacks

Advantages

- fast

Drawbacks

PCA advantages and drawbacks

Advantages

- fast
- scales well

Drawbacks

PCA advantages and drawbacks

Advantages

- fast
- scales well
- explanatory

Drawbacks

PCA advantages and drawbacks

Advantages

- fast
- scales well
- explanatory

Drawbacks

- not suited for non-linear data

Non linear data

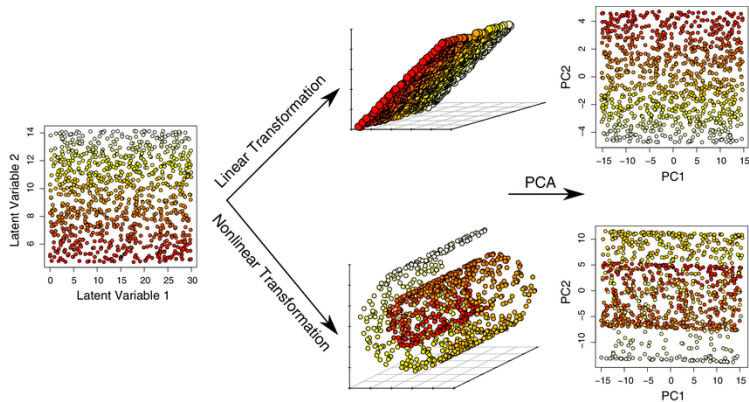


Figure from Du, 2019

Projections

UMAP

How to work with non linear data

1. Find a good representation of the data in high dimension
2. Fit a good representation of in low dimension

- **SNE** (Stochastic Neighbor Embedding) Hinton and Roweis, 2002

Classics

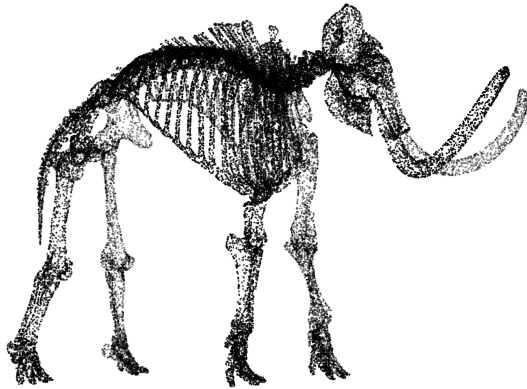
- **SNE** (Stochastic Neighbor Embedding) Hinton and Roweis, 2002
- **T-SNE** (T-distributed SNE) Van der Maaten and Hinton, 2008

Classics

- **SNE** (Stochastic Neighbor Embedding) Hinton and Roweis, 2002
- **T-SNE** (T-distributed SNE) Van der Maaten and Hinton, 2008
- **UMAP** (Uniform Manifold Approximation and Projection) McInnes et al., 2018

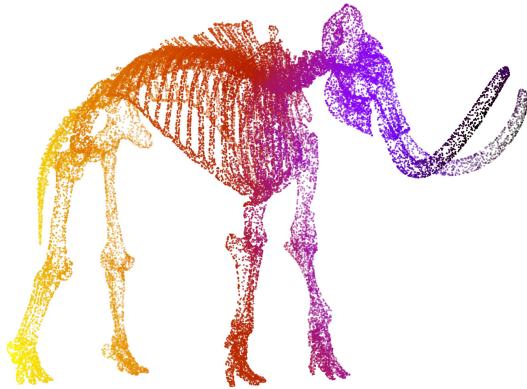
How does UMAP work ?

Mammoth example



Mammoth dataset by the Smithsonian

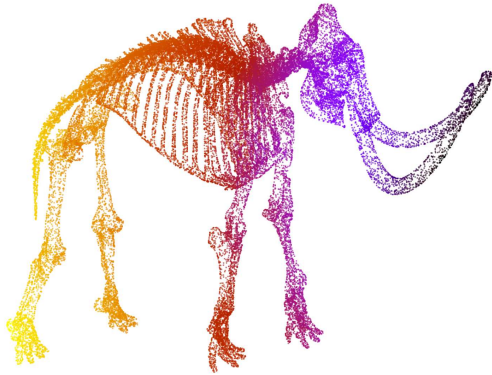
Mammoth example



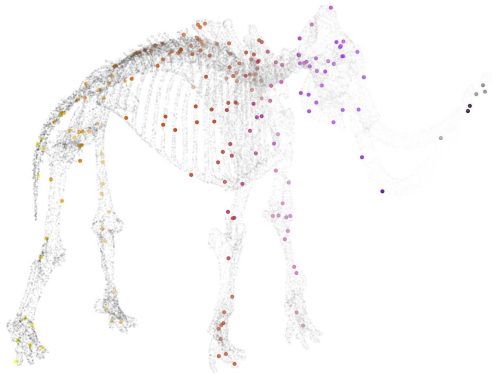
Mammoth dataset by the Smithsonian



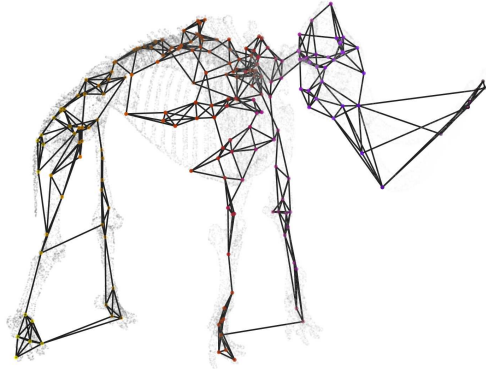
Finding high dimension graph



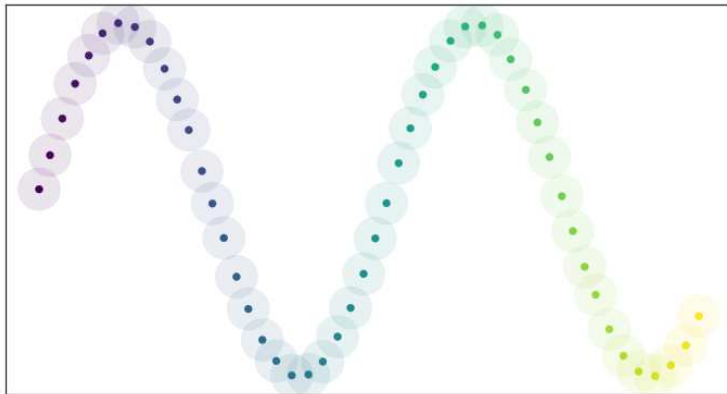
Finding high dimension graph



Finding high dimension graph

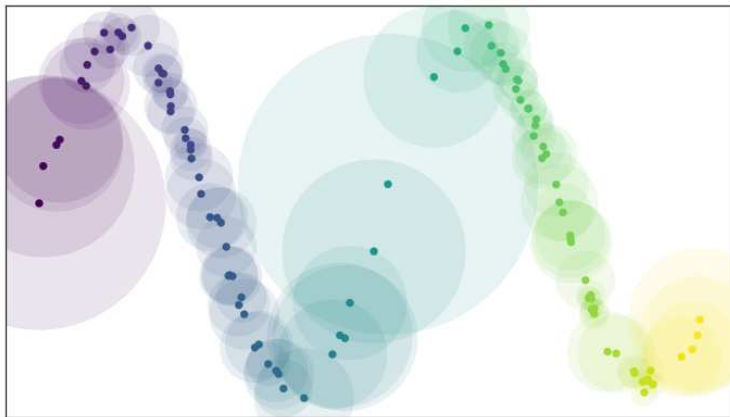


Uniform Manifold Approximation and Projection



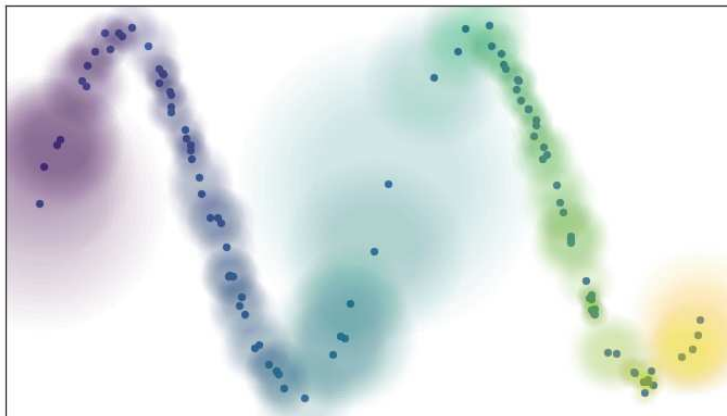
[UMAP documentation](#)

Uniform Manifold Approximation and Projection



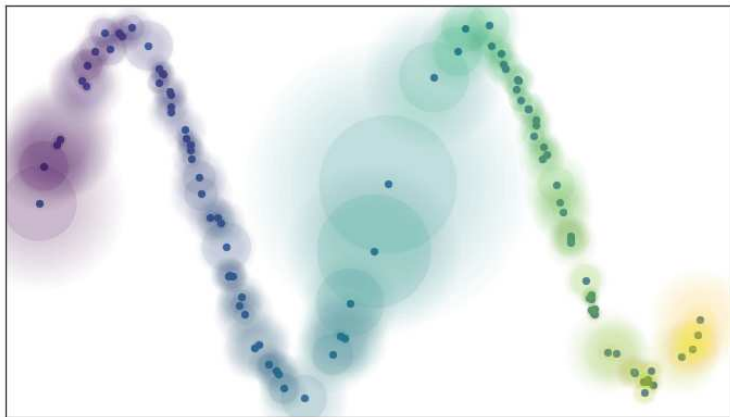
UMAP documentation

Uniform Manifold Approximation and Projection



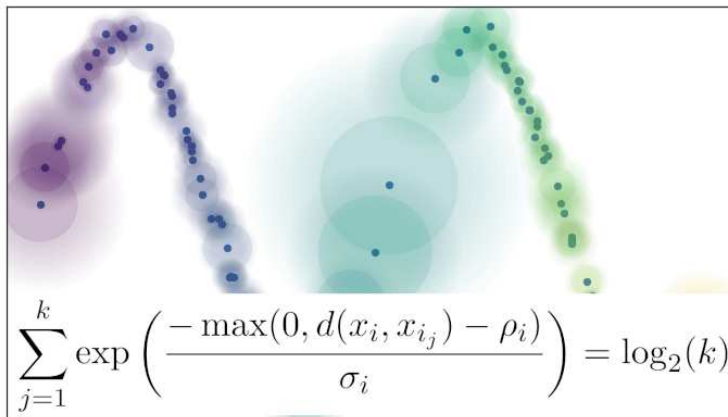
[UMAP documentation](#)

Uniform Manifold Approximation and Projection



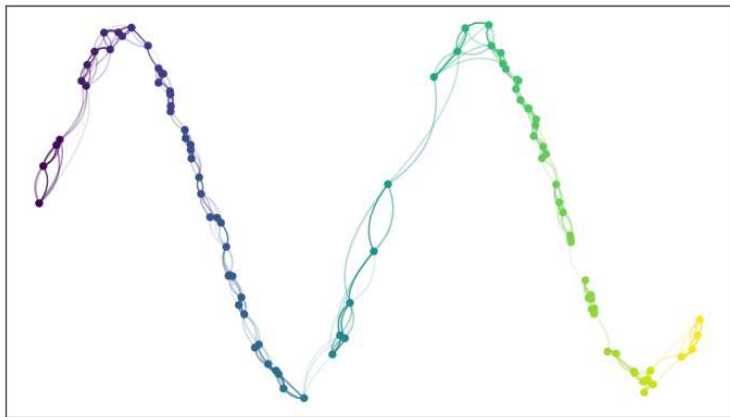
[UMAP documentation](#)

Uniform Manifold Approximation and Projection



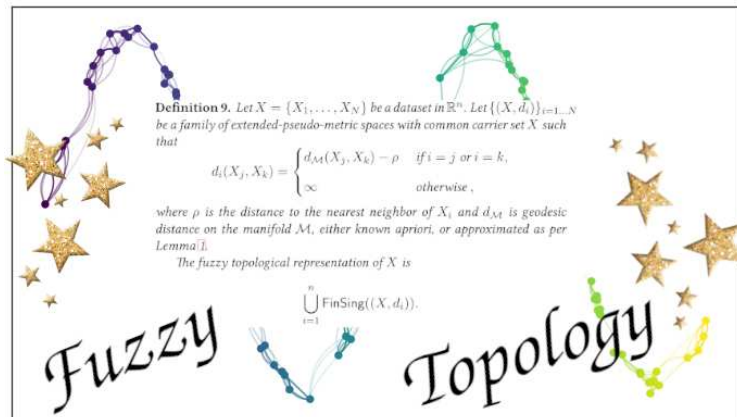
UMAP documentation

Uniform Manifold Approximation and Projection

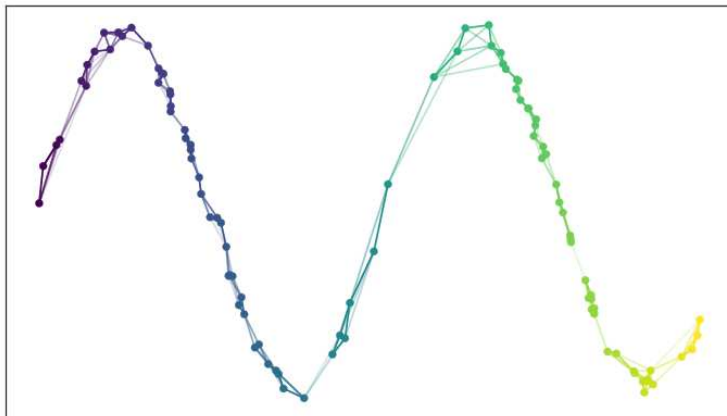


[UMAP documentation](#)

Uniform Manifold Approximation and Projection

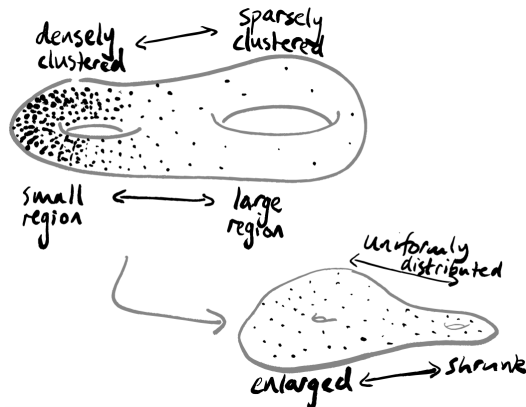


Uniform Manifold Approximation and Projection

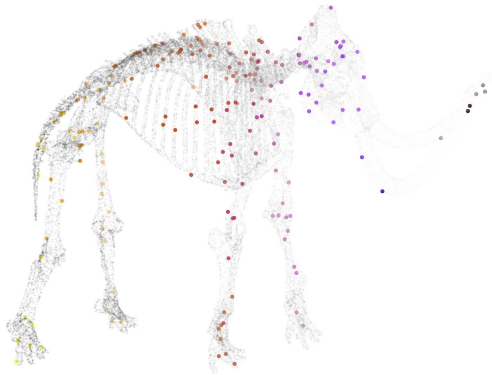


[UMAP documentation](#)

Uniform Manifold Approximation and Projection



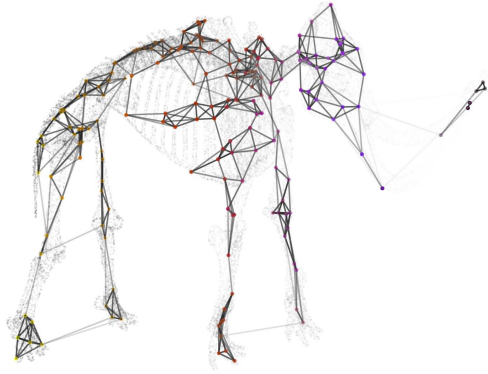
Finding high dimension graph



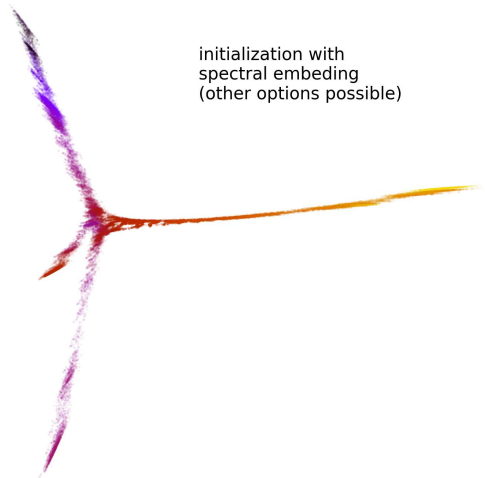
Finding high dimension graph



Fitting low dimensional representation

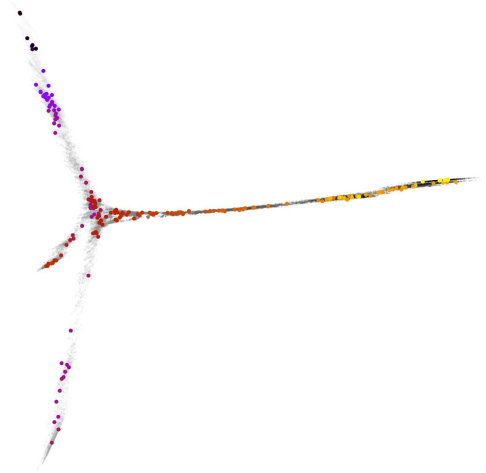


Fitting low dimensional representation



initialization with
spectral embedding
(other options possible)

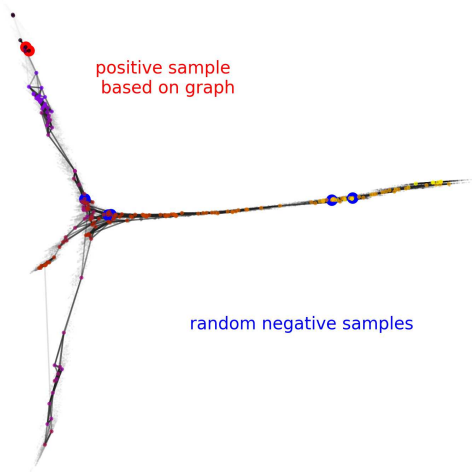
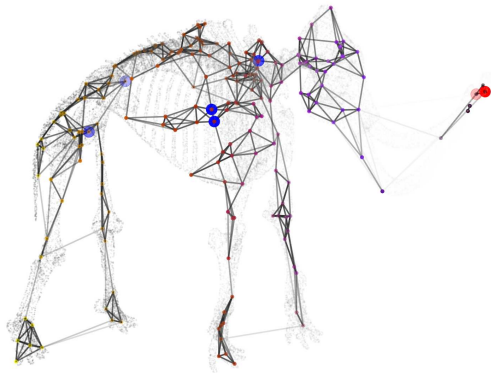
Fitting low dimensional representation



Fitting low dimensional representation



Fitting low dimensional representation



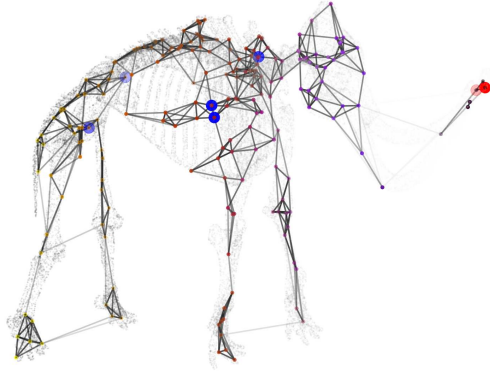
Fitting low dimensional representation

Cross-entropy loss counting **positive** and **negative** attraction forces

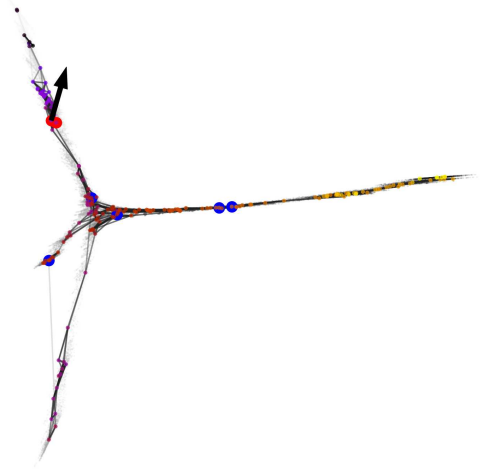
$$C_{UMAP} = \sum_{i \neq j} v_{ij} \log \left(\frac{v_{ij}}{w_{ij}} \right) + (1 - v_{ij}) \log \left(\frac{1 - v_{ij}}{1 - w_{ij}} \right) \quad (1)$$

Optimization with **Stochastic Gradient Descent**

Fitting low dimensional representation



Fitting low dimensional representation

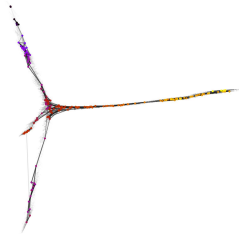


Fitting low dimensional representation

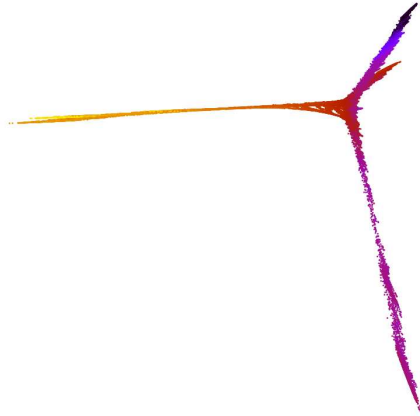


	point 1	point 2	point 3	point 4	point 5...
point 1	0	0.13	0.023	0.64	0.045
point 2	0.13	0	0.92	0	0.45
point 3	0.023	0.92	0	0.22	0.47
point 4	0.64	0	0.22	0	0.61
point 5...	0.045	0.45	0.47	0.61	0

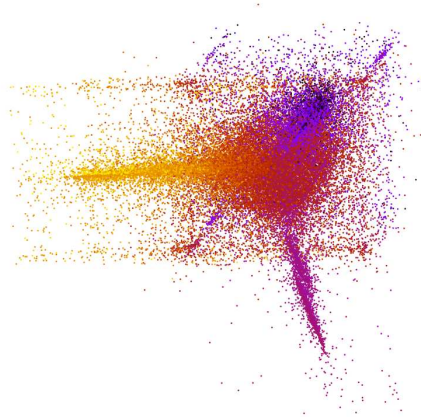
	point 1	point 2	point 3	point 4	point 5...
point 1	0	0.28	0.89	0.59	0.41
point 2	0.28	0	0.047	0.58	0.55
point 3	0.89	0.047	0	0.49	0.60
point 4	0.59	0.58	0.49	0	0.47
point 5...	0.41	0.55	0.60	0.47	0



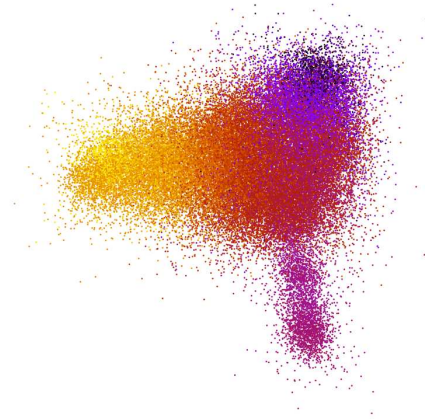
Optimization



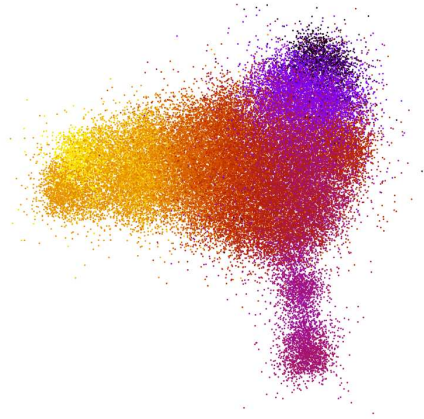
Optimization



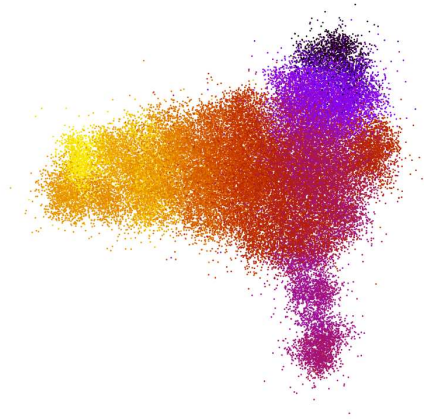
Optimization



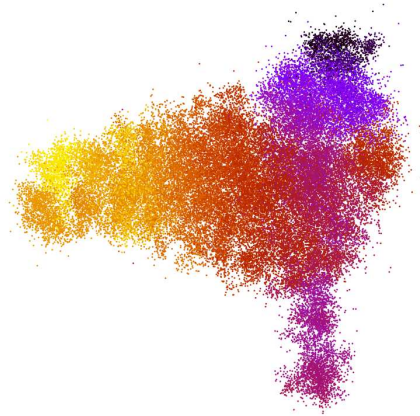
Optimization



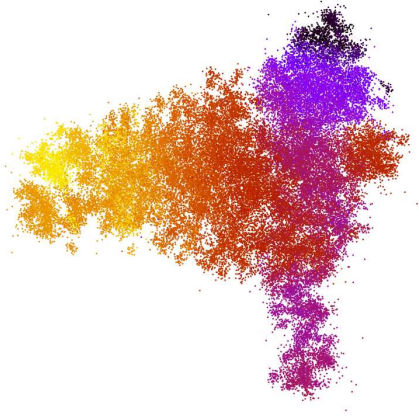
Optimization



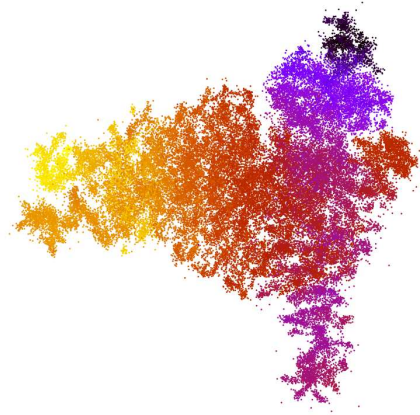
Optimization



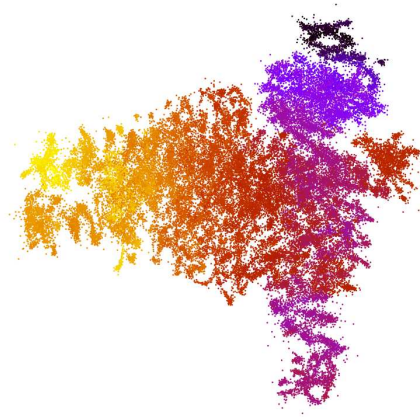
Optimization



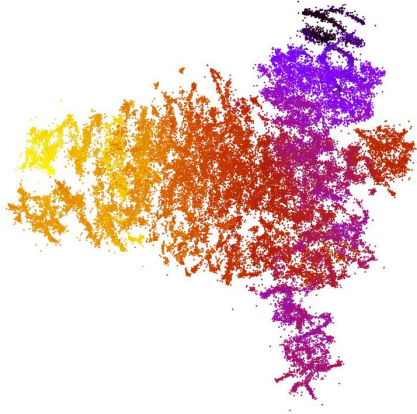
Optimization



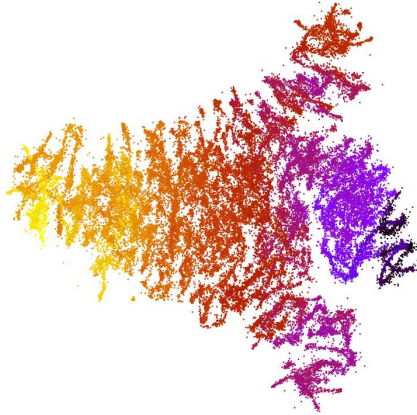
Optimization



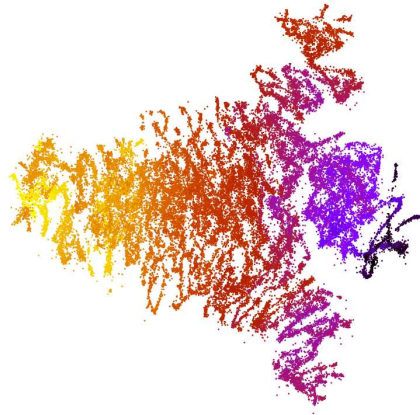
Optimization



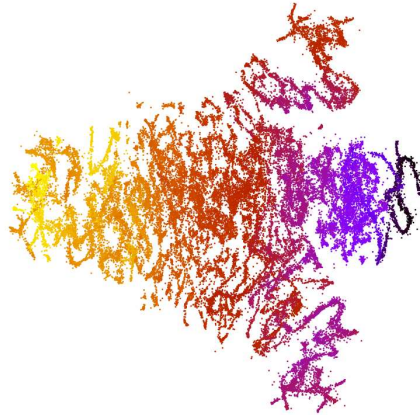
Optimization



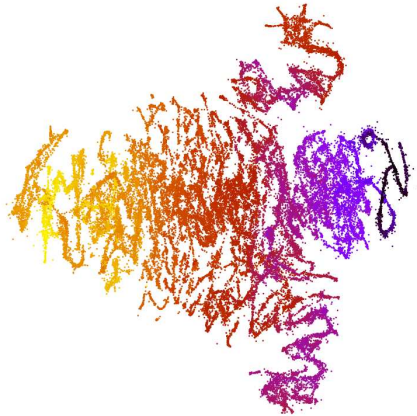
Optimization



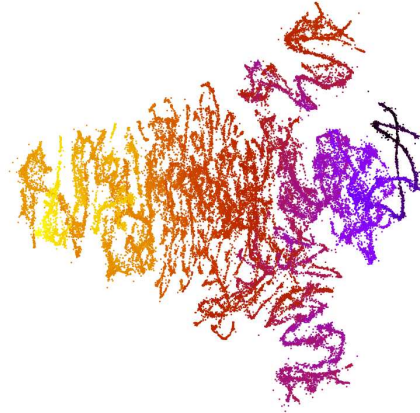
Optimization



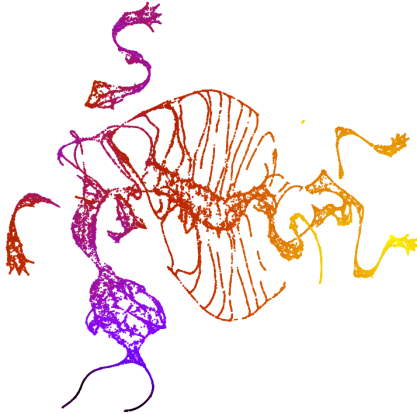
Optimization



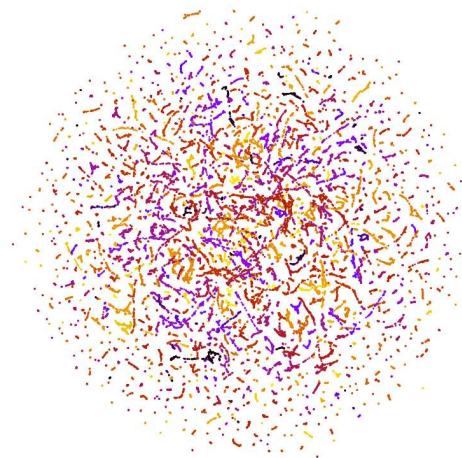
Optimization



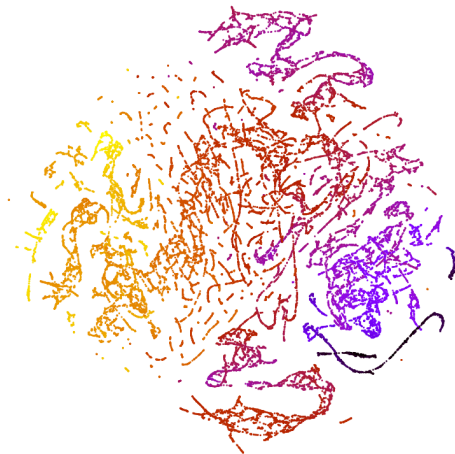
Optimization



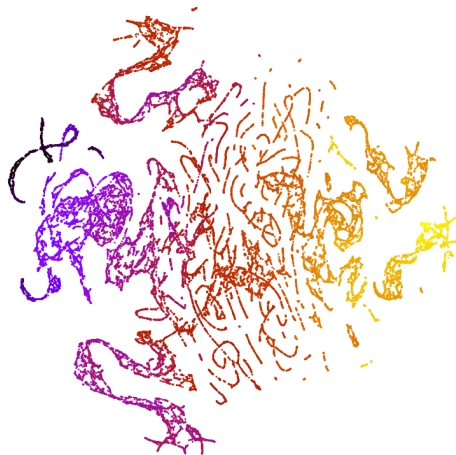
Mammoth example - UMAP



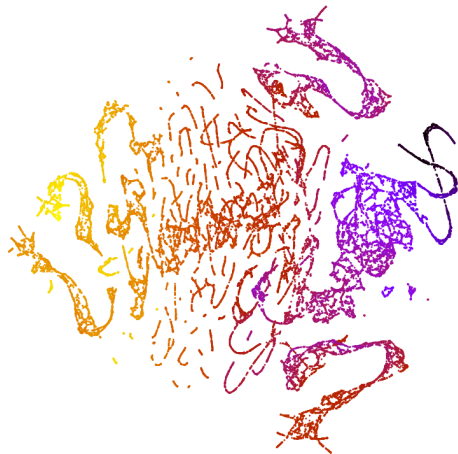
Mammoth example - UMAP



Mammoth example - UMAP



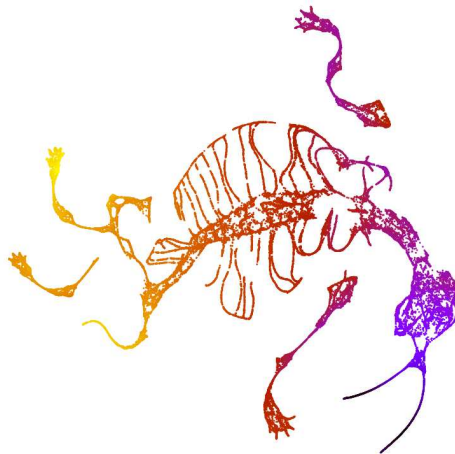
Mammoth example - UMAP



Mammoth example - UMAP



Mammoth example - UMAP



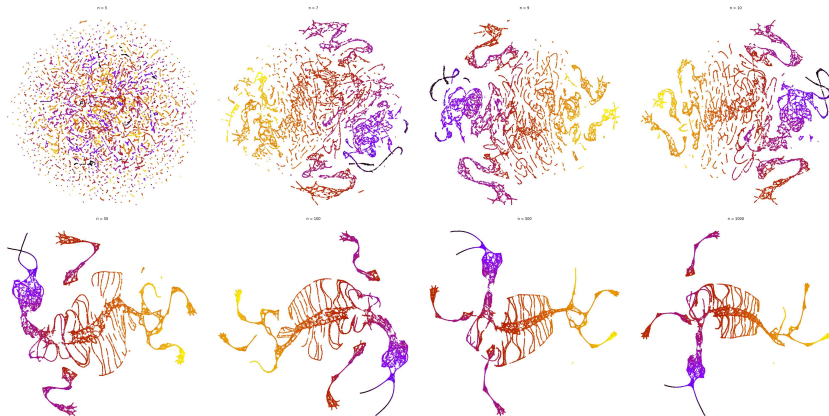
Mammoth example - UMAP



Mammoth example - UMAP



Mammoth example - Number of neighbors



UMAP vs. T-SNE

UMAP

- graph theory, fuzzy logic

T-SNE

- probabilities

UMAP vs. T-SNE

UMAP

- graph theory, fuzzy logic
- deterministic initialization

T-SNE

- probabilities
- random initialization

UMAP vs. T-SNE

UMAP

- graph theory, fuzzy logic
- determinist initialization
- $\log_2$ similarity scores

T-SNE

- probabilities
- random initialization
- gaussian distribution similarity

UMAP vs. T-SNE

UMAP

- graph theory, fuzzy logic
 - determinist initialization
 - $\log_2$ similarity scores
 - update by pairs
- scales well for large datasets

T-SNE

- probabilities
 - random initialization
 - gaussian distribution similarity
 - update all points
- scales less

Scaling

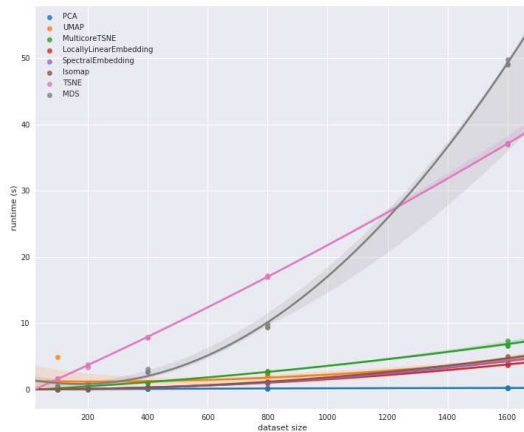


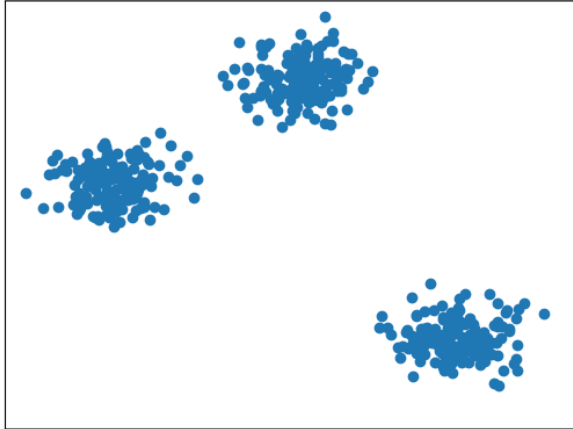
Figure from umap-learn documentation

Clustering

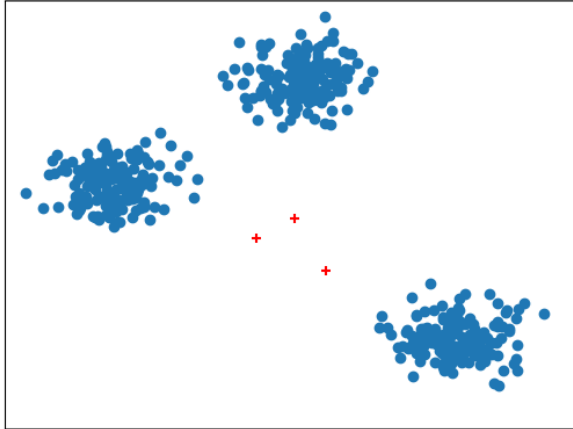
Clustering

K-means

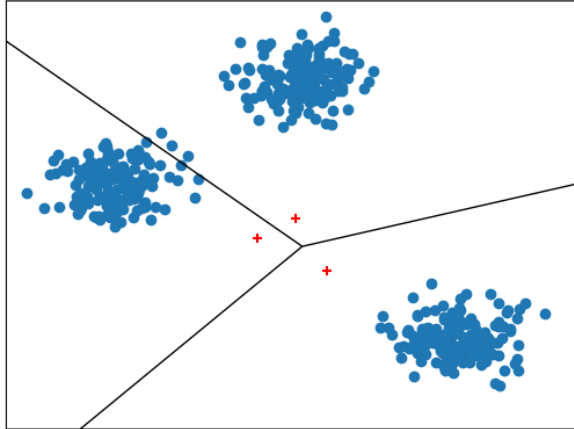
K-means



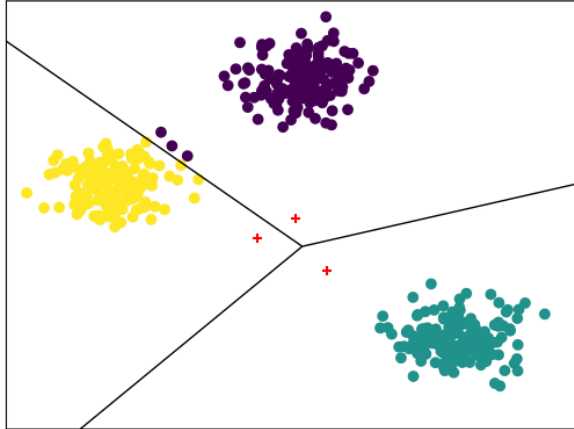
K-means



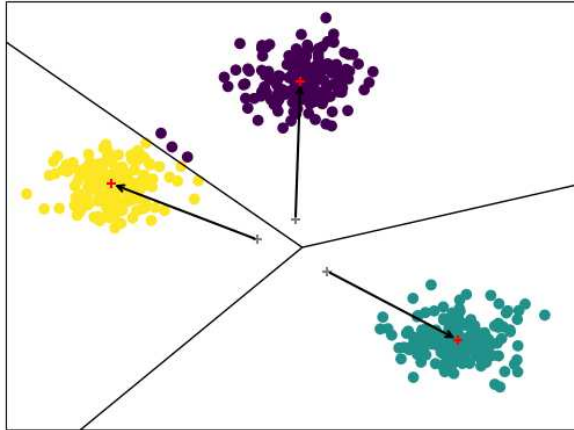
K-means



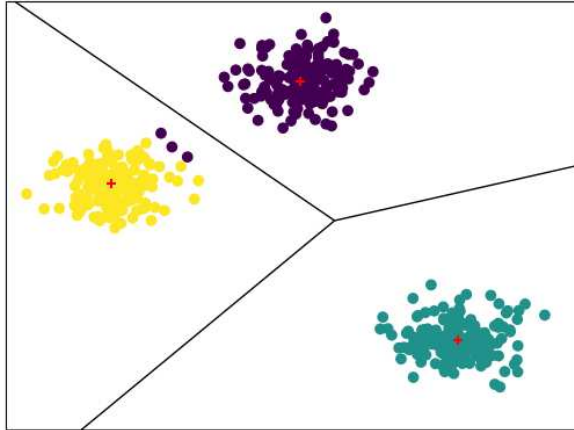
K-means



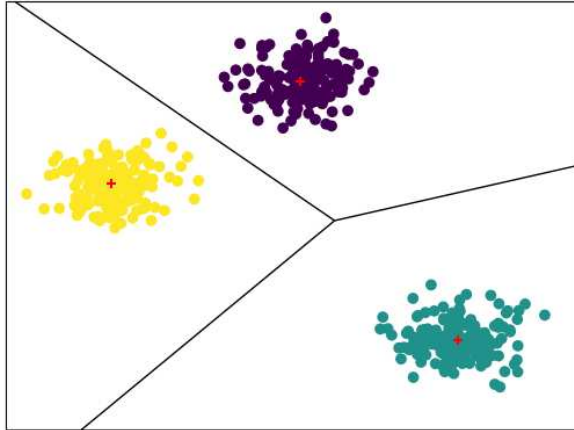
K-means



K-means



K-means



Determine the good K ? Elbow method

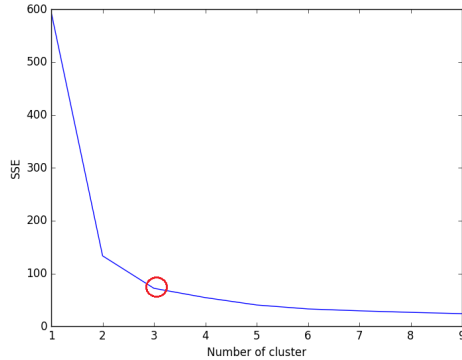
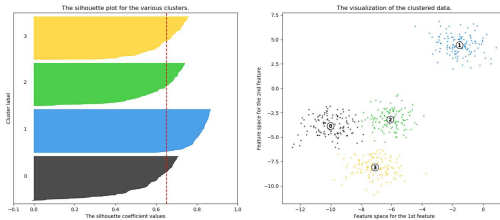


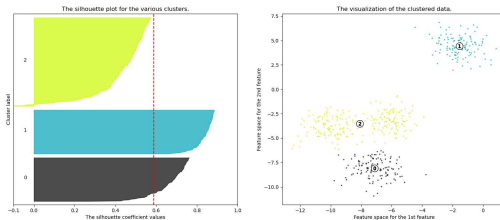
Figure from SO

Determine the good K ? Silhouette

Silhouette analysis for KMeans clustering on sample data with $n_clusters = 4$



Silhouette analysis for KMeans clustering on sample data with $n_clusters = 3$



Figures from scikit-learn documentation

K-means advantages and drawbacks

Advantages

- fast

Drawbacks

K-means advantages and drawbacks

Advantages

- fast
- scales well

Drawbacks

K-means advantages and drawbacks

Advantages

- fast
- scales well
- converges well

Drawbacks

K-means advantages and drawbacks

Advantages

- fast
- scales well
- converges well
- robust

Drawbacks

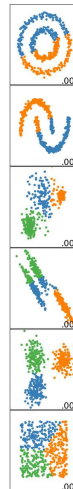
K-means advantages and drawbacks

Advantages

- fast
- scales well
- converges well
- robust

Drawbacks

- not suited for non-linear data



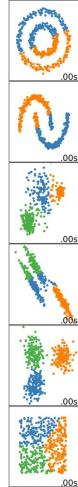
K-means advantages and drawbacks

Advantages

- fast
- scales well
- converges well
- robust

Drawbacks

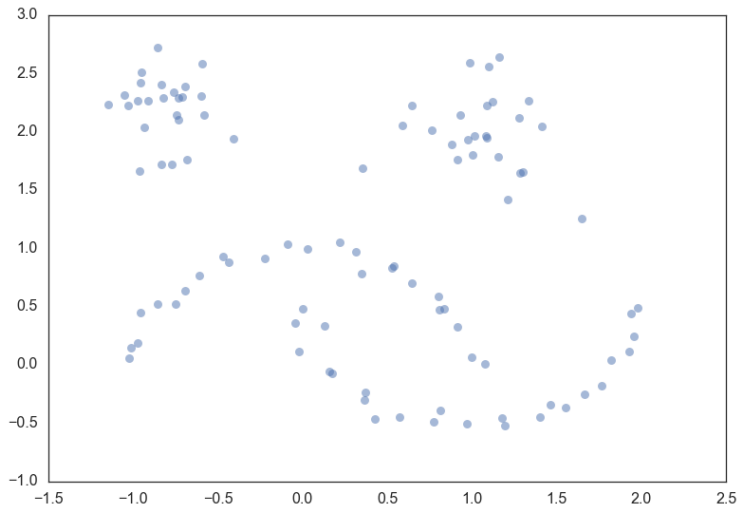
- not suited for non-linear data
- Density based clustering



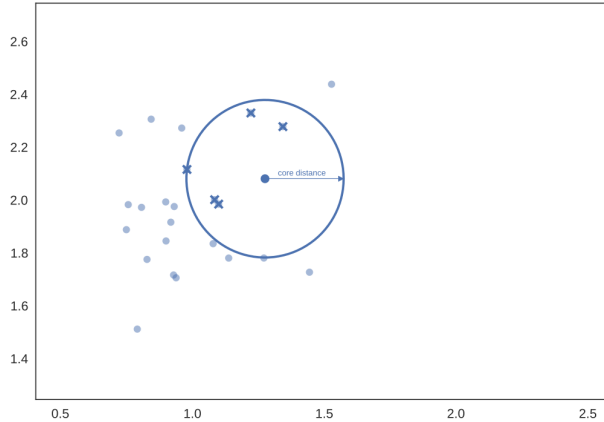
Clustering

HDBSCAN

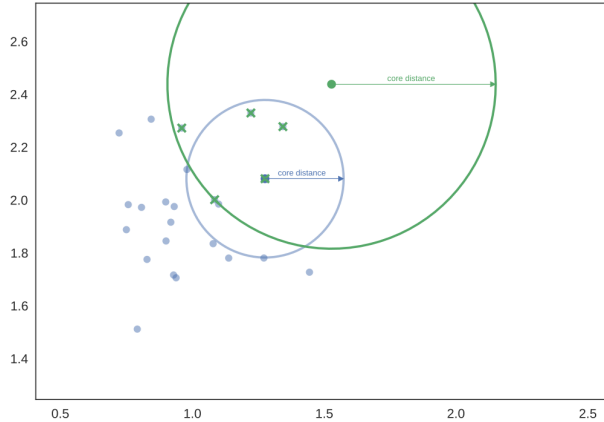
HDBSCAN



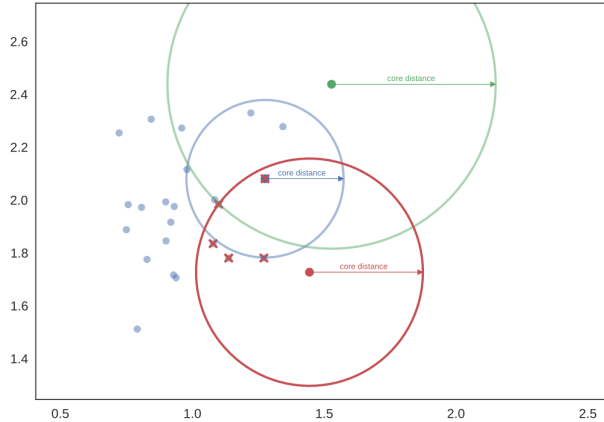
HDBSCAN



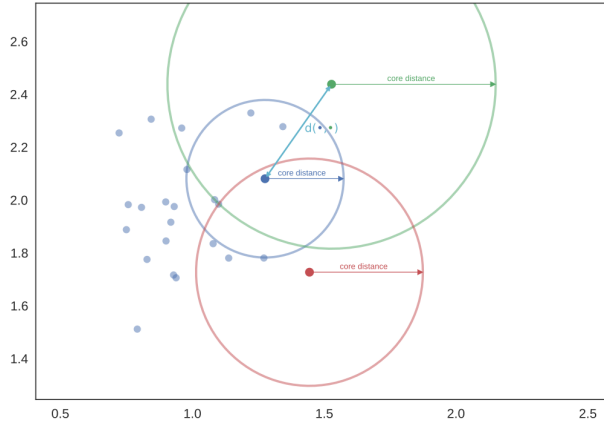
HDBSCAN



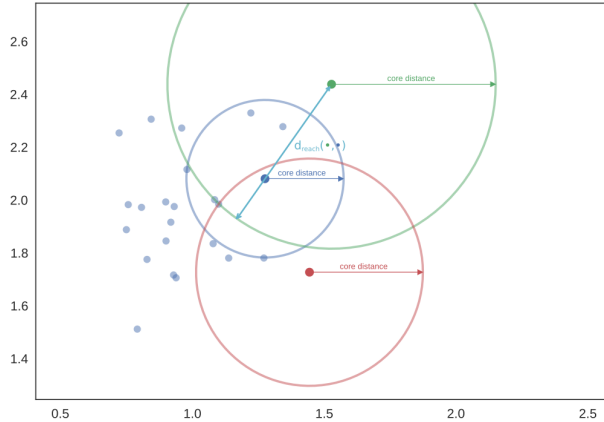
HDBSCAN



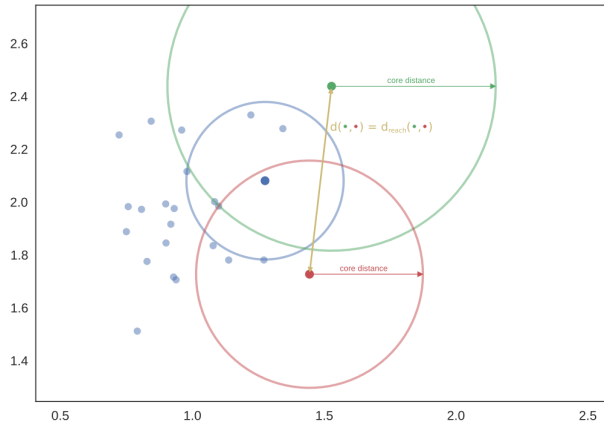
HDBSCAN



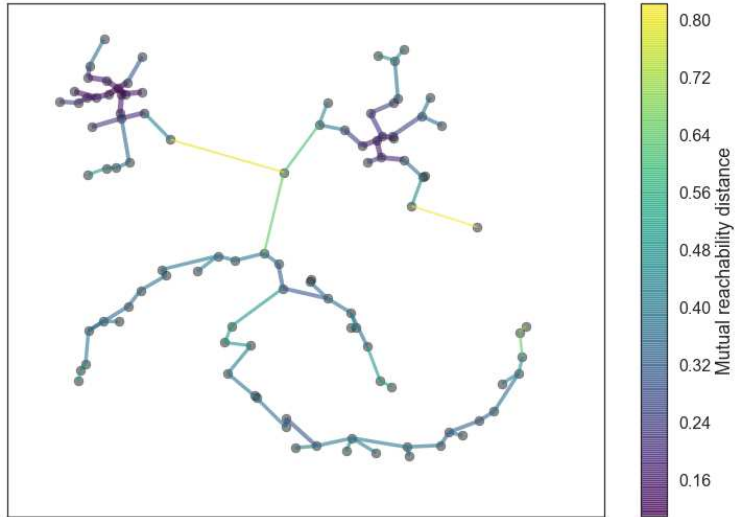
HDBSCAN



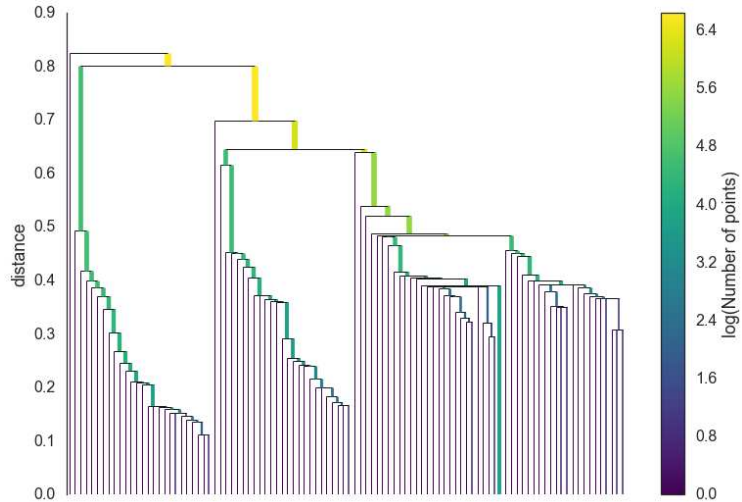
HDBSCAN



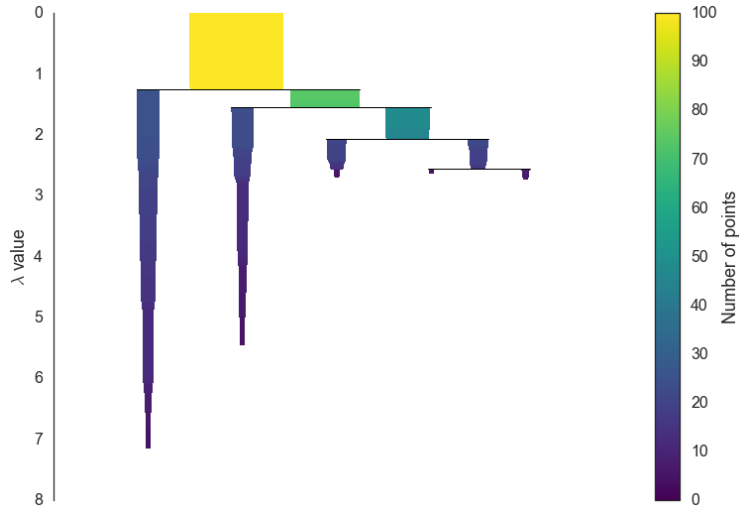
HDBSCAN



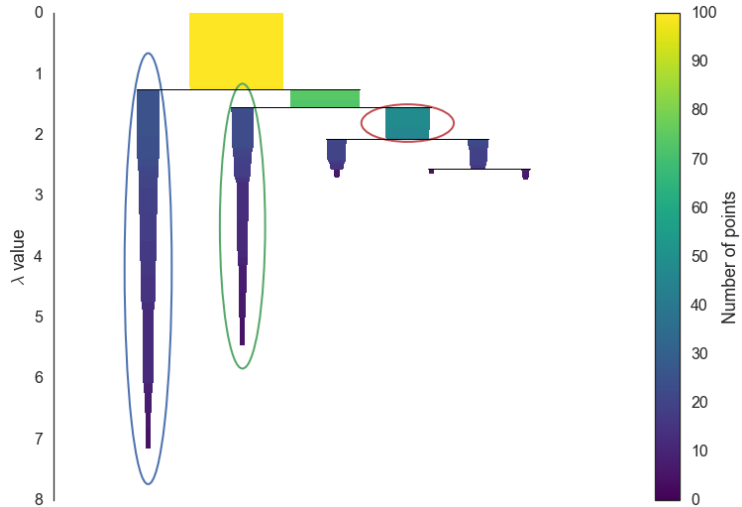
HDBSCAN



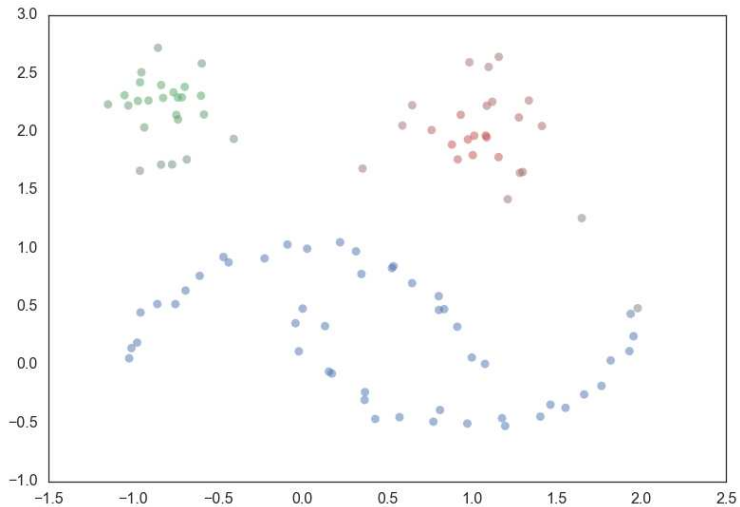
HDBSCAN



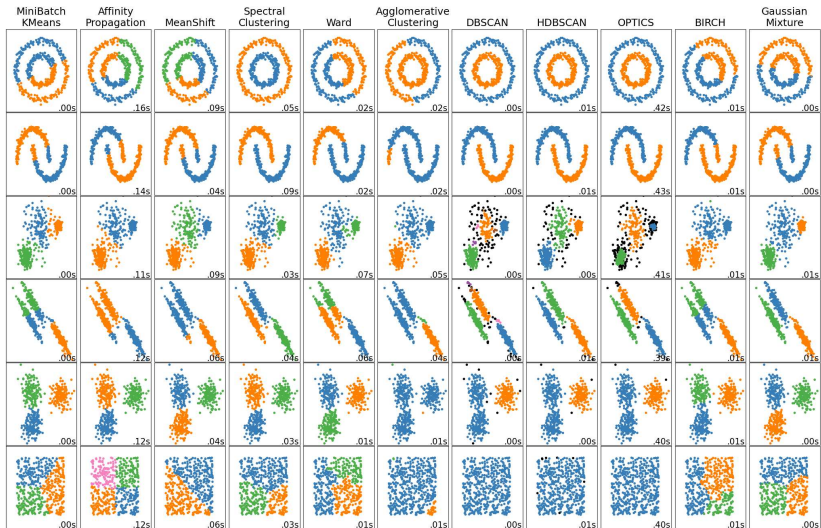
HDBSCAN



HDBSCAN



A lot of options !



Useful ressources

- `scikit-learn` docs !
- deepia
- StatQuest

Thanks for you attention !

Let's practice !

References i

- Du, Trina Y (2019). **“Dimensionality reduction techniques for visualizing morphometric data: Comparing principal component analysis to nonlinear methods”**. In: *Evolutionary Biology* 46.1, pp. 106–121.
- Hinton, Geoffrey E and Sam Roweis (2002). **“Stochastic neighbor embedding”**. In: *Advances in neural information processing systems* 15.
- McInnes, Leland, John Healy, and James Melville (2018). **“Umap: Uniform manifold approximation and projection for dimension reduction”**. In: *arXiv preprint arXiv:1802.03426*.
- Van der Maaten, Laurens and Geoffrey Hinton (2008). **“Visualizing data using t-SNE.”**. In: *Journal of machine learning research* 9.11.